

Universidade Tecnológica Federal do Paraná

Engenharia de Software - Curso de Arquitetura de Software (AS27S)

INSTRUTOR: Prof. Dr. Gustavo Santos

Aluno Juan Gabriel Benitez Bogo, RA 2601753

CCH - Design Patterns Proxy

Link do projeto:

<https://github.com/juaNGb01/Trabalho-Arquitetura---Designs-Patterns.git>

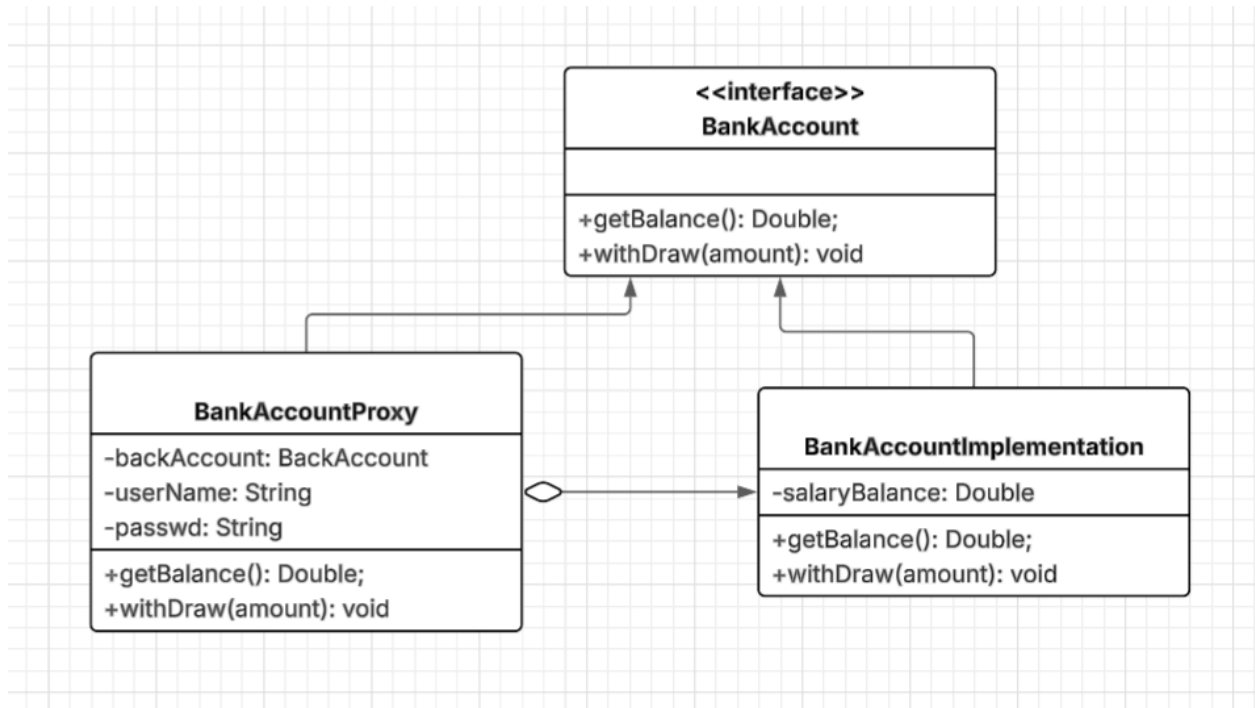
Problema

Criar um programa para controle de uma conta bancária, na qual qualquer usuário pode visualizar o saldo dessa conta, mas apenas o usuário com a senha correta poderá realizar operações de saque. Porém como realizar isso sem aplicar as verificações diretamente na classe que implementa uma conta bancária

Descrição da Solução

Com a utilização do padrão proxy é possível criar uma espécie de classe verificação/escudo que irá intermediar as operações. Com a classe BankAccountProxy criamos uma classe que implementa a mesma interface usada na classe concreta da conta, garantindo que tenhamos duas classes praticamente iguais, contudo o usuário interage diretamente com o proxy e não com a classe conta bancária. Após o proxy realizar as verificações ele irá chamar os métodos de saque da classe concreta

Visão Geral [exemplo]



Exemplo de Código [linguagem de programação da sua preferência]

```
public interface BankAccount {

    Double getBalance();

    void withdraw(double amount);

}

public class BankAccountImplementation implements BankAccount {
```

```
private Double salaryBalance;

public BankAccountImplementation(Double salary) {

    this.salaryBalance = salary;

}

@Override

public Double getBalance() {

    return salaryBalance;

}

@Override

public void withdraw(double amount) {

    if (amount <= salaryBalance) {

        salaryBalance = salaryBalance - amount;

        System.out.println("Retirada confirmada, saldo atual:" +
getBalance());

    } else {

        System.out.println("Saldo insuficiente!");

    }

}
```

```
}  
  
}
```

```
package org.example.proxy;  
  
import org.example.BankAccountInterface.BankAccount;  
  
import org.example.implementation.BankAccountImplementation;  
  
public class BankAccountProxy implements BankAccount {  
  
    private BankAccount backAccount;  
  
    private String userName;  
  
    private String passwd;  
  
    public BankAccountProxy(BankAccount backAccount, String userName,  
String passwd) {  
  
        this.backAccount = backAccount;  
  
        this.userName = userName;  
  
        this.passwd = passwd;  
  
    }  
}
```

```
@Override

public Double getBalance() {

    return backAccount.getBalance();

}

@Override

public void withdraw(double amount) {

    if(userName.equals("Megan") && passwd.equals("4002")){

        backAccount.withdraw(amount);

    }else{

        System.out.println("Usuário " + userName + " não tem
acesso a essa operação");

    }

}

}
```

Consequências [vantagens e desvantagens]

-
- Maior segurança. É possível garantir um melhor controle de acesso a conta bancária, se novas políticas de acesso precisam ser implementadas basta alterar apenas na classe do proxy
 - Código mais limpo. Com a implementação desse padrão a classe principal da conta bancária não precisa lidar com o controle de acesso, é repassado essa função a classe do proxy que irá a classe concreta apenas quando as credenciais estiverem válidas
 - É possível criar diferentes tipos de proxies um lidando com uma coisa diferente, por exemplo um proxy que lida com o controle de login na conta e outro que lida com as consultas de saldo e isso pode ser feito sem realizar alterações diretas na classe principal